Generics

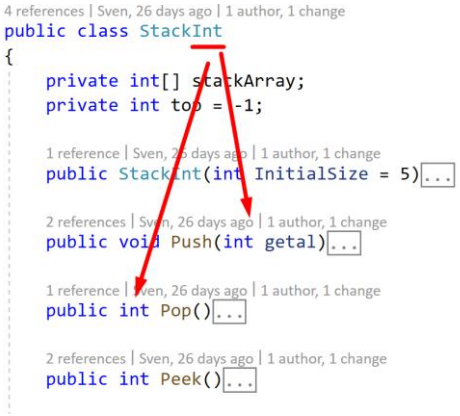
Bachelor IT

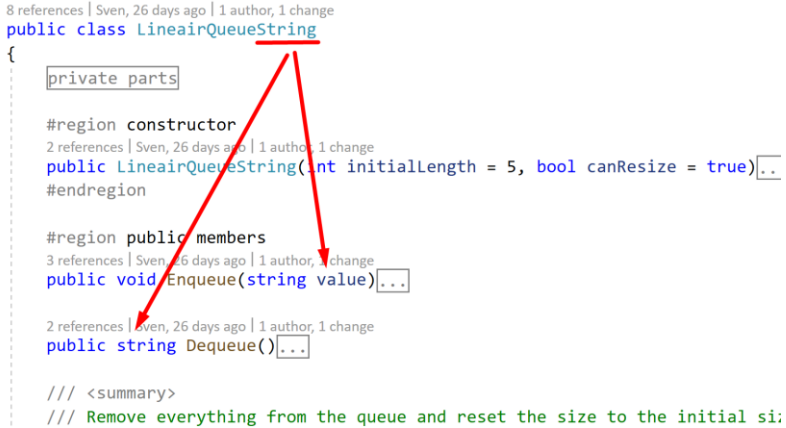
AP HOGESCHOOL ANTWERPEN

AP.BE

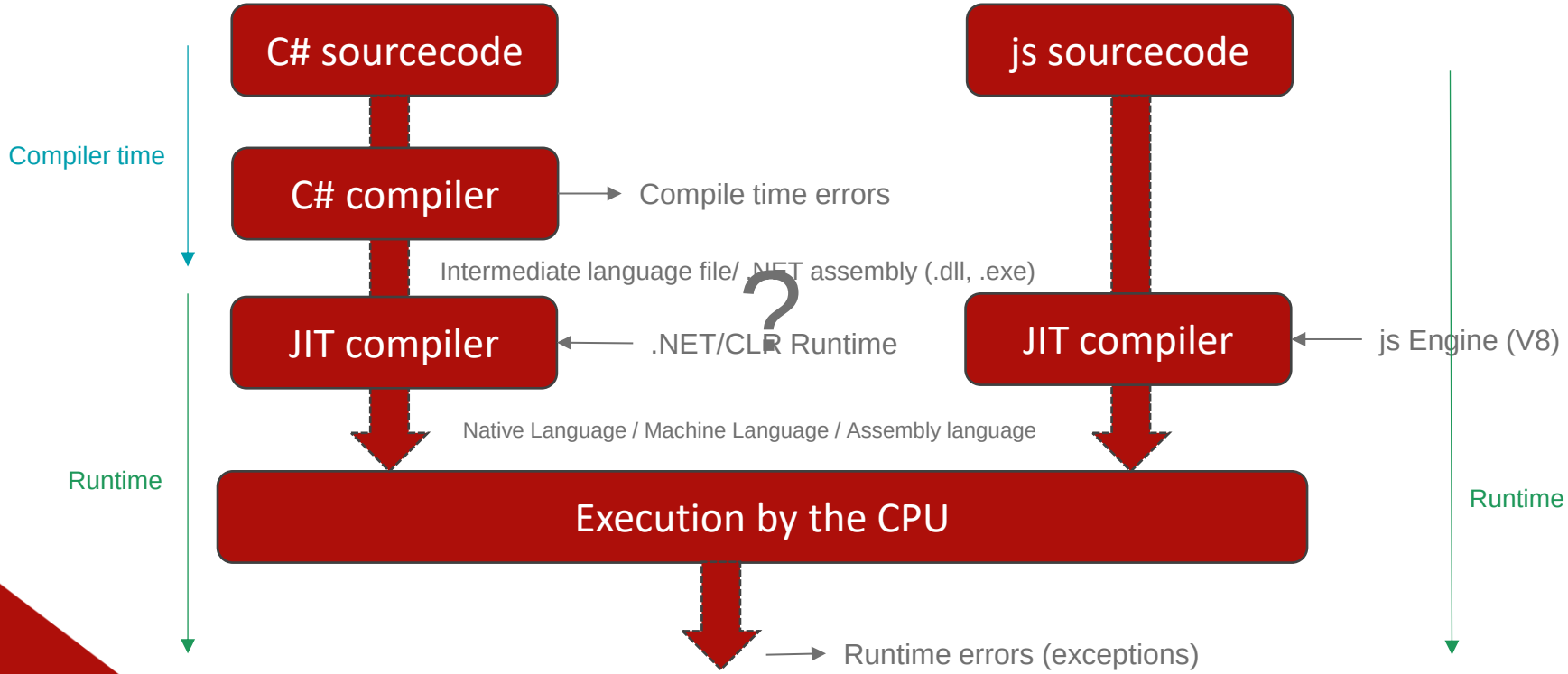
Datastructures

- Queue & Stack
- Sorteeralgoritmen
- Linked List & Tree
- Zoekalgoritmen



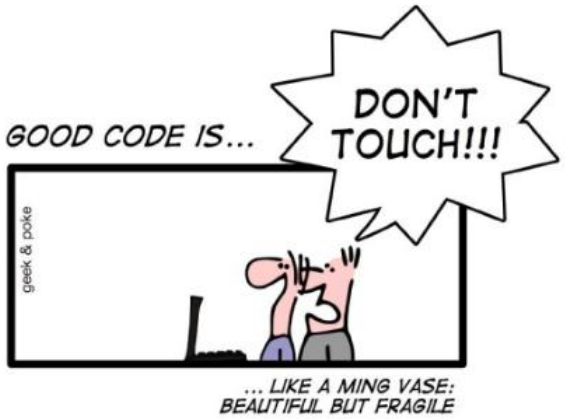


C# versus javascript



Compiler of geen compiler…

- **Compiler errors** brengen een probleem **onmiddellijk** aan het licht tegenover **run-time exceptions** die pas optreden **als de code** daadwerkelijk **wordt uitgevoerd**.
- Beeld je in dat je een applicatie hebt met 10.000 lijnen broncode, dwz. dat je elke lijn steeds moet testen als je iets hebt aangepast in de code.



C# en js: verschil in typesystems

- C# werkt
  - met een “**static type system**”
  - dwz. dat **elke variabele of parameter** een **vast type** heeft.
  - Dat type ligt vast van bij de **declaratie**
- js werkt
  - met een “**dynamic type system**”
  - dwz. dat het **type** van een variable **kan wijzigen** tijdens de levensduur.
  - Het **type** kan zelfs “**undefined**” zijn…

Types: c# vs. javascript

C#

```
int getal = 10;
string woord = "hallo";
bool vlag = true;
var teller = 29;        //Type inference (afleiden van het type adhv. de initële waarde)

getal = "hallo";        //this is not js ..
```

js

```
var a;
console.log(`a has value:${a} & is now type: '${typeof(a)}'`);
a = 10
console.log(`a has value:${a} & is now type: '${typeof(a)}'`);
a = "DATASTRUCTURES"
console.log(`a has value:${a} & is now type: '${typeof(a)}'`);
a = true
console.log(`a has value:${a} & is now type: '${typeof(a)}'`);
```

```
a has value:undefined & is now type: 'undefined'
a has value:10 & is now type: 'number'
a has value:DATASTRUCTURES & is now type: 'string'
a has value:true & is now type: 'boolean'
```

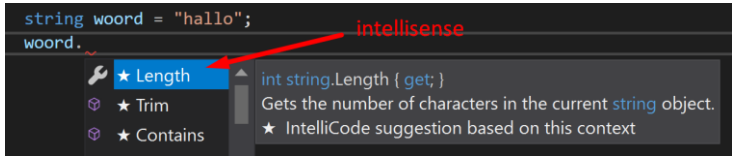
C#

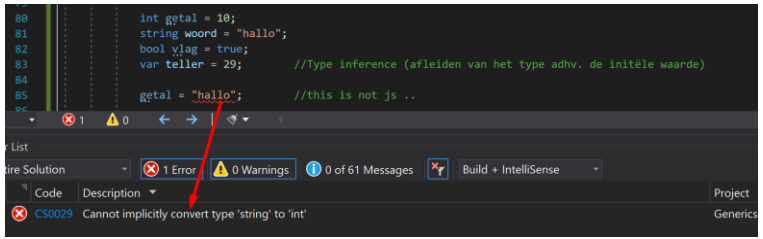
```
object watzouikhiermeedoen = "Help";

dynamic weGaanDeJavascriptToerOp = 33;
weGaanDeJavascriptToerOp = "Dit kan dus echt ook in c# :-)";
```

Static type system

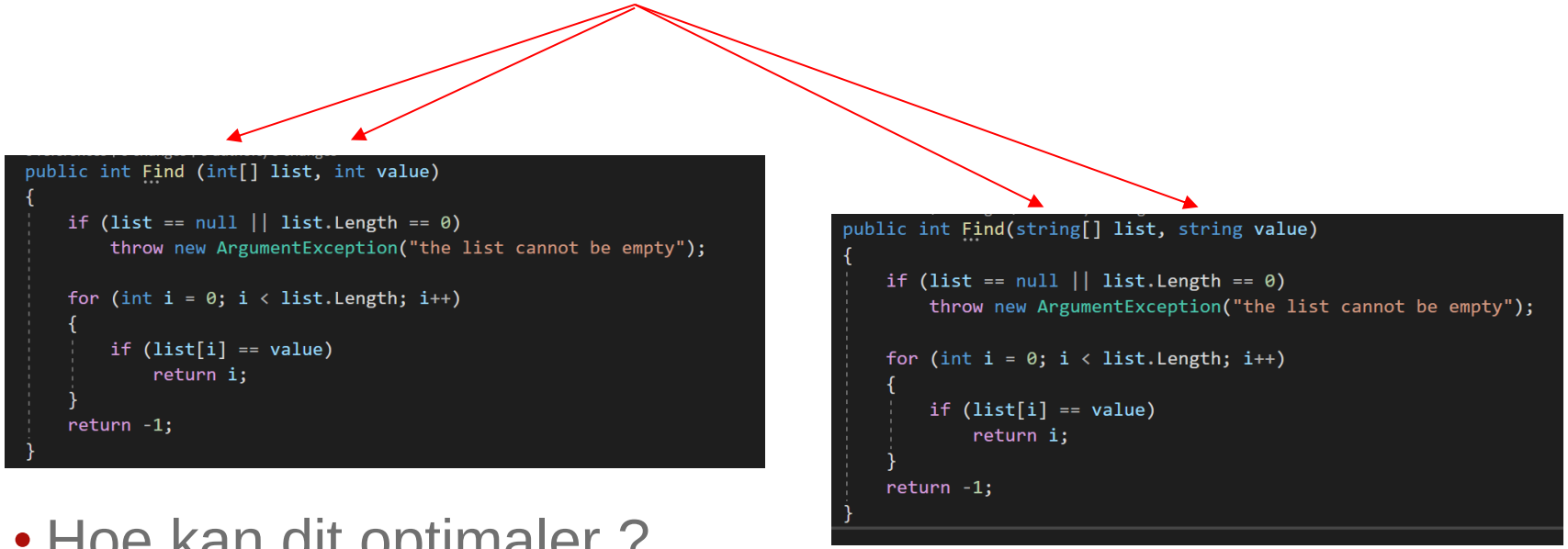
- Werken met static types:
  - **Intellisense:** Je weet “at development time” exact welke properties en methodes je kan aanroepen.
  - **Type safety:** Compiler zal fouten geven als je de applicatie compileert (opstart).





c#: Maximaal broncode hergebruiken

- Als we code vergelijken voor bv. een **lineair search**:



```int
public int Find (int[] list, int value)
{
    if (list == null || list.Length == 0)
        throw new ArgumentException("the list cannot be empty");

    for (int i = 0; i < list.Length; i++)
    {
        if (list[i] == value)
            return i;
    }
    return -1;
}
```

```string
public int Find(string[] list, string value)
{
    if (list == null || list.Length == 0)
        throw new ArgumentException("the list cannot be empty");

    for (int i = 0; i < list.Length; i++)
    {
        if (list[i] == value)
            return i;
    }
    return -1;
}
```

- Hoe kan dit optimaler ?

Broncode hergebruiken => type object ?

- Laten we starten met een eenvoudigere methode.

```
public int[] CreateArray(int item1, int item2)
{
    int[] array = new int[2];
    array[0] = item1;
    array[1] = item2;
    return array;
}
```

```
public string[] CreateArray(string item1, string item2)
{
    string[] array = new string[2];
    array[0] = item1;
    array[1] = item2;
    return array;
}
```

?

```
public object[] CreateArray(object item1, object item2)
{
    object[] array = new object[2];
    array[0] = item1;
    array[1] = item2;
    return array;
}
```

```
string a = "AP";
string b = "Hogeschool";

Test t = new Test();
object[] result = t.CreateArray(a, b);

if (result[1].Length > 5)
{
    //...
}
```

Compiler error

CS1061: 'object' does not contain a definition for 'Length' and no accessible extension method 'Length' accepting a first argument of type 'object' could be found (are you missing a using directive or an assembly reference?)

Broncode hergebruiken => type dynamic ?

```
public int[] CreateArray(int item1, int item2)
{
    int[] array = new int[2];
    array[0] = item1;
    array[1] = item2;
    return array;
}
```

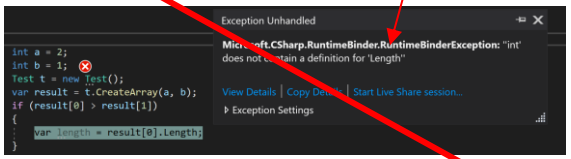
```
public string[] CreateArray(string item1, string item2)
{
    string[] array = new string[2];
    array[0] = item1;
    array[1] = item2;
    return array;
}
```

?

```
public dynamic[] CreateArray(dynamic item1, dynamic item2)
{
    dynamic[] array = new dynamic[2];
    array[0] = item1;
    array[1] = item2;
    return array;
}
```

```
int a = 1;
int b = 2;
Test t = new Test();
var result = t.CreateArray(a, b);
if (result[0] > result[1])
{
    var length = result[0].Length;
}
```

Runtime error



Optie 3: **Generics** !

- We schrijven de code 1x
- De code kan worden hergebruikt voor meerdere types
- De performantie is beter (ook geen casting nodig)
- De types toch ook worden “at compile time” geverifieerd.
- We hebben dus ook intellisense !

Use generic types to maximize code reuse, type safety and performance !

Generics: generieke methode maken

```
public int[] CreateArray(int item1, int item2)
{
    int[] array = new int[2];
    array[0] = item1;
    array[1] = item2;
    return array;
}
```

```
public string[] CreateArray(string item1, string item2)
{
    string[] array = new string[2];
    array[0] = item1;
    array[1] = item2;
    return array;
}
```

?

```
public T[] CreateArray<T>(T item1, T item2)
{
    T[] array = new T[2];
    array[0] = item1;
    array[1] = item2;
    return array;
}
```

```
int a = 2;
int b = 1;
Test t = new Test();
var result = t.CreateArray<int>(a, b);
if (result[0] > result[1])
{
    var length = result[0].Length;
}
```

Generics: generieke methode aanroepen

```
public T[] CreateArray<T>(T item1, T item2)
{
    T[] array = new T[2];
    array[0] = item1;
    array[1] = item2;
    return array;
}
```

```
int a = 2;
int b = 1;
Test t = new Test();
var result = t.CreateArray<int>(a, b);
if (result[0] > result[1])
{
    var length = result[0].Length;
}
```

```
string a = "Hallo";
string b = "AP";
Test t = new Test();
var result = t.CreateArray<string>(a, b);
if (result[0] > result[1])
{
    var length = result[0].Length;
}
```

```
Auto auto = new Auto()
{
    AantalKm = 10000,
    Bouwjaar = 2005,
    Brandstof = Brandstof.Waterstof,
    Kleur = "Rood",
    Model = "Ferrari F40"
};

Auto auto2 = new Auto()
{
    AantalKm = 20000,
    Bouwjaar = 2005,
    Brandstof = Brandstof.Electriciteit,
    Kleur = "Groen",
    Model = "Ferrari F40"
};

Test t = new Test();
var result = t.CreateArray<Auto>(auto, auto2);
if (result[1].B)
```

Bouwjaar
Brandstof
int Auto.Bouwjaar { g

Compile time type checking

Generic Stack bouwen

```
public class StackInt
{
    private int[] stackArray;
    private int top = -1;

    1 reference | Sven, 13 days ago | 1 author, 1 change
    public StackInt(int InitialSize = 5)
    {
        stackArray = new int[InitialSize];
    }

    2 references | Sven, 13 days ago | 1 author, 1 change
    public void Push(int getal)
    {
        if (IsFull)
            throw new Exception("Sorry the stack is full");

        stackArray[++top] = getal;
    }

    1 reference | Sven, 13 days ago | 1 author, 1 change
    public int Pop()
    {
        if (IsEmpty)
            throw new Exception("The stack is empty. Pop is not allowed");

        return stackArray[top--];
    }
```

```
public class Stack<T>
{
    private T[] stackArray;
    private int top = -1;

    3 references | Sven, 1 day ago | 1 author, 1 change
    public Stack(int InitialSize = 5)
    {
        stackArray = new T[InitialSize];
    }

    6 references | Sven, 1 day ago | 1 author, 1 change
    public void Push(T item)
    {
        if (IsFull)
        {
            var newArray = new T[stackArray.Length * 2];
            System.Array.Copy(stackArray, newArray, stackArray.Length);
            stackArray = newArray;
        }

        stackArray[++top] = item;
    }

    1 reference | Sven, 1 day ago | 1 author, 1 change
    public T Pop()
    {
        if (IsEmpty)
            throw new Exception("The stack is empty. Pop is not allowed");

        return stackArray[top--];
    }
```

Generic stack gebruiken

```
public class Stack<T>
{
    private T[] stackArray;
    private int top = -1;

    3 references | Sven, 1 day ago | 1 author, 1 change
    public Stack(int InitialSize = 5)
    {
        stackArray = new T[InitialSize];
    }

    6 references | Sven, 1 day ago | 1 author, 1 change
    public void Push(T item)
    {
        if (IsFull)
        {
            var newArray = new T[stackArray.Length * 2];
            System.Array.Copy(stackArray, newArray, stackArray.Length);
            stackArray = newArray;
        }

        stackArray[++top] = item;
    }

    1 reference | Sven, 1 day ago | 1 author, 1 change
    public T Pop()
    {
        if (IsEmpty)
            throw new Exception("The stack is empty. Pop is not allowed");

        return stackArray[top--];
    }
```

```
0 references | 0 changes | 0 authors, 0 changes
static void Main(string[] args)
{
    Console.WriteLine("Hello World!");

    var stack = new Stack<int>(20);
    stack.Push(10);
    stack.Push(12);
    stack.Push(5);

    int last = stack.Pop();
    //.....

    var stack2 = new Stack<double>(10);
    stack2.Push(10.34);
    stack2.Push(22.99);
    //...

    var stack3 = new Stack<Auto>(20);
    stack3.Push(new Auto());
    //...
}
```